

計算機科学概論

第11回

塚田 武志 ・ 山本 光晴

2025.12.26 千葉大学

今回の目標

アセンブラ

アセンブラ

- 背景
- 演習で作るアセンブラの仕様
- 実装について

アセンブラ

- **背景**
- 演習で作るアセンブラの仕様
- 実装について

アセンブラ

アセンブリ言語で書かれたプログラムを **0/1 で書かれた機械語**に変換

- アセンブリの命令と機械語は おおむね 一対一に対応

例

- | | |
|---------------------|----------------------------|
| • M=D+M; JLE | 1111 0000 1000 1110 |
| • 0; JMP | 1110 1010 1000 0111 |
| • @1024 | 0000 0010 0000 0000 |

- ひとつのやっかいな問題は**シンボル**でメモリを参照すること

例

- **@SCREEN**

シンボル

メモリアドレスや命令アドレスを表す記号

- アドレスは数だが，それを表す記号があると便利

例

- ループの先頭の命令のアドレスに **LOOP** という記号を対応させる
- プログラムの終わり部分のアドレスに **END** という記号を対応させる
- 「変数 i」を格納するメモリアドレスに **i** という記号を対応させる

シンボルの典型的な使用方法是次の2つ

- **変数** 変換器が**自動で割当てる**メモリアドレス（値は重要でない）
- **ラベル** プログラム中の**特定の位置**を表すシンボル

シンボル解決

シンボルに適切な値を割り当てること

```
@100
D=A
@i
M=D
@sum
M=0
(LOOP)
@i
D=M
@end
D;JLE
@sum
M=D+M
@LOOP
0;JMP
(END)
0;JMP
```

シンボル解決前の
プログラム

```
@100
D=A
@16
M=D
@17
M=0

@16
D=M
@14
D;JLE
@17
M=D+M
@6
0;JMP

0;JMP
```

i	16
sum	17
LOOP	6
END	14

シンボルテーブル

シンボル解決後の
プログラム

本講義における シンボル解決のルール

- 変換されたコードは,
ROM のアドレス 0 を
先頭に配置される
- 変数の格納場所は,
RAM のアドレス 16 から

※ 本講義のハードウェアは
命令とデータを別の記憶素子に持つので
重複がなく簡単

※ こうしたルールは,
ハードウェアに依存して
適切に与えられる

```
@100
D=A
@i
M=D
@sum
M=0
(LLOOP)
@i
D=M
@END
D;JLE
@sum
M=D+M
@LOOP
0;JMP
(END)
0;JMP
```

i	16
sum	17
LOOP	6
END	14

シンボルテーブル

シンボル解決前の
プログラム

アセンブラの処理内容

1. アセンブリプログラムを構文解析して，命令ごとに分割
2. 命令中のシンボルを数値に置換
3. 命令ごとに，対応する機械語の 0/1 の列を生成
4. 生成された機械語を組み合わせて，機械語プログラムを作成

このうち 1, 3, 4 は比較的容易

シンボルの処理（2）は，これらよりも難しい

- アセンブリの主要な機能のひとつ

アセンブラ

- 背景
- 演習で作るアセンブラの仕様
- 実装について

アセンブリの仕様

A命令 @数値 または @シンボル

- Aレジスタに数値またはシンボルに対応する値を格納

C命令 **dest=comp;jump**

- **dest** 計算結果の出力先. 空のときは = を省略
- **comp** 計算内容
- **jump** ジャンプする条件. 空のときは ; を省略

ラベル定義 (シンボル)

- この次の命令のアドレスをシンボルに割当てる

C命令の詳細

<u>dest</u>	<u>comp</u>			<u>jump</u>
null	0	D+1	M	null
M	1	A+1	!M	JGT
D	-1	D-1	-M	JEQ
MD	D	A-1	M+1	JGE
A	A	D+A	M-1	JLT
AM	!D	D-A	D+M	JNE
AD	!A	A-D	D-M	JLE
AMD	-D	D&A	D&M	JMP
	-A	D A	D M	

アセンブリのその他の仕様

- 命令の前後の空白文字は無視される
- 「//」で行末までコメント
- 空行（空白とコメントだけからなる行）は無視される
- 次のシンボルは最初から定義されている

SP	0
LCL	1
ARG	2
THIS	3
THAT	4
SCREEN	16384
KBD	24576

R0	0
R1	1
R2	2
...	
R15	15

機械語命令の仕様

A命令 0XXX XXXX XXXX XXXX

- Aレジスタに 0XXX XXXX XXXX XXXX を格納

C命令 1??a cccc ccdd djjj

- ? 無意味なビット（実は必ず 1 だが気にしないでよい）
- a ALUへの入力はAレジスタかメモリか（メモリ = 1）
- c 演算内容を指定する． ALU の演算指定 6 bit に対応
- d ALU の演算結果を書き込む対象を指定（書き込む = 1）
左から Aレジスタ/Dレジスタ/メモリ
- j ALU の演算結果に応じてジャンプするか（ジャンプする = 1）
左から 負/ゼロ/正 の時に

参考： ALU の仕様

f(x,y)	zx	nx	zy	ny	f	no
0	1	0	1	0	1	0
1	1	1	1	1	1	1
-1	1	1	1	0	1	0
x	0	0	1	1	0	0
y	1	1	0	0	0	0
!x	0	0	1	1	0	1
!y	1	1	0	0	0	1
-x	0	0	1	1	1	1
-y	1	1	0	0	1	1
x+1	0	1	1	1	1	1
y+1	1	1	0	1	1	1
x-1	0	0	1	1	1	0
y-1	1	1	0	0	1	0
x+y	0	0	0	0	1	0
x-y	0	1	0	0	1	1
y-x	0	0	0	1	1	1
x&y	0	0	0	0	0	0
x y	0	1	0	1	0	1

アセンブラ

- 背景
- 演習で作るアセンブラの仕様
- **実装について**

実装のための参考プログラム

asm.py

- Python で書かれたアセンブラの構文解析器
- コマンド行から「`python3 asm.py [ファイル名]`」で実行できる
 - ファイルからアセンブリプログラムを読み，構文解析する
 - ファイル名を省略したときは標準入力から読む
 - 構文解析結果は，`Command` の列
 - `SymbolDecl` / `ACommand` / `CCommand` クラスのインスタンス
 - どの種類のコマンドかは `isinstance` を使えば分かる
- **Google Colaboratory** での実行の仕方は授業中に説明
- 配布時の動作：構文解析して，コマンド列を出力

参考

- 2^{15} より小さい非負整数を15桁の二進数（文字列）に変換するには
`format(num, "015b")`
- 文字列の連結は `+`
- 二進文字列への変換や文字列の連結は `f-string` を使用して行うことも可能
- 辞書（連想配列）`dict` の鍵に `key` があるかの判定は
`key in dict`
- 辞書 `dict` へ鍵 `key` で情報 `data` の項目の追加は
`dict[key] = data`

実装の手順

1. C命令の命令とビット列の変換表を作成
 - `dest_table`, `comp_table`, `jump_table` の **None** を適切な文字列に
2. シンボルがないプログラムの処理を与える
 - A命令で格納すべき値は，必ず数で書かれる
3. シンボル定義はしないが，最初からあるシンボルは使う場合の処理
 - A命令で必要に応じて `symbol_table` を読む必要がある
4. 最初のシンボル表にないシンボルも用いる場合の処理

1. C命令の変換表

```
dest_table: Dict[str, str] = {  
    "null": "000",  
    "M": "001",  
    "D": None,  
    "MD": None,  
    "A": None,  
    "AM": None,  
    "AD": None,  
    "AMD": None  
}
```

```
comp_table: Dict[str, str] = {  
    "0": "0101010",  
    "1": "0111111",  
    "-1": None,  
    "D": None,  
    "A": None,  
    "!D": None,  
    "!A": None,  
    "-D": None,  
    "-A": None,  
    "D+1": None,  
    "A+1": None,  
    "D-1": None,  
    "A-1": None,
```

2. シンボルのない命令の変換

```
if isinstance(c, ACommand):  
    print(「c.argを2進数15桁で書いて、先頭に0を加えたもの」)  
  
elif isinstance(c, CCommand):  
    print("111" + 「c.comp に対応する0/1列」  
          + 「c.dest に対応する0/1列」  
          + 「c.jump に対応する0/1列」 )  
  
else:  
    raise AssembleError(c.line)
```

3. 最初からあるシンボルを使う場合

- A命令の処理に変更が必要
- A命令 `c` に対して `c.is_int_argument()` で引数が数か否かが分かる
- もし引数が数でない（＝シンボル）であるなら，
`symbol_table` を参照してAレジスタに格納する値を決める

4. シンボル定義を含む場合

この場合は一回のループで解決するのは難しい

命令全体を見るループ **for c in cmds** を2回行う

- 一周目はラベル定義を解決（変数定義は放って置く）
 - `linenum` を 0 から始めて, A命令/C命令を見るたびにひとつ増やす
 - 命令がラベル定義 `SymbolDecl` だった場合は次の処理を行う
 1. 定義しようとしているシンボル `c.name` が `symbol_table` において未定義であることの確認
 2. `symbol_table` の鍵 `c.name` の値を `linenum` とする
- 二周目は変数定義を解決しつつ命令を機械語に変換
 - A命令でシンボルを見たら, 定義済みシンボルかを確認
 - もし定義済でなければ新しい変数.
シンボルの値に `var_num` をセットして, `var_num` をインクリメント

課題11

締切：2026/01/23 23:59 (JST)

課題11

- A) 実装手順 1 を行え
- B) 実装手順 2 を行え
- C) 実装手順 3 を行え
- D) 実装手順 4 を行え

それぞれ必要なプログラムの書き換えを行い、
その内容をレポートで説明せよ

なお各手順に対応するテストプログラムがあるので活用されたい